

Лабораторная работа

Тема «Изменение, добавление и удаление HTML-элементов и их свойств с помощью сценариев на языке JavaScript»

Цель: научиться использовать сценарии JavaScript для создания, добавления, изменения и удаления HTML-элементов и их свойств.

JavaScript предоставляет ряд методов для управления элементами на веб-страницы. В частности, мы можем создавать и добавлять новые элементы или заменять и удалять уже имеющиеся. Рассмотрим эти методы.

Создание элементов

Для создания элементов объект `document` имеет следующие методы:

- **`createElement(elementName)`**: создает элемент html, тег которого передается в качестве параметра. Возвращает созданный элемент
- **`createTextNode(text)`**: создает и возвращает текстовый узел. В качестве параметра передается текст узла.

Создадим элемент с помощью **`createElement`**:

```
const header = document.createElement("h1"); // создаем заголовок <h1>
console.log(header); // <h1></h1>
```

Таким образом, переменная `header` будет хранить ссылку на элемент `h1`.

Создадим текстовый узел с помощью **`createTextNode`**:

```
const headerText = document.createTextNode("Hello World"); // создаем
текстовый узел
console.log(headerText); // "Hello World"
```

Добавление элементов

Однако одного создания элементов недостаточно, их еще надо добавить на веб-страницу.

Для добавления элементов мы можем использовать один из методов объекта `Node`:

- **`appendChild(newNode)`**: добавляет новый узел `newNode` в конец коллекции дочерних узлов
- **`insertBefore(newNode, referenceNode)`**: добавляет новый узел `newNode` перед узлом `referenceNode`

`appendChild`

Используем метод **`appendChild()`**:

```
const header = document.createElement("h1"); // создаем заголовок <h1>
const headerText = document.createTextNode("Hello World"); // создаем
текстовый узел
header.appendChild( headerText); // добавляем в элемент h1 текстовый узел
console.log(header); // <h1>Hello World</h1>
```

И чтобы добавить созданный элемент на страницу, его надо добавить в уже имеющийся на странице элемент:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Добавление элементов (append)</title>
</head>
<body>
  <script>
    const header = document.createElement("h1"); // создаем заголовок <h1>
    const headerText = document.createTextNode("Hello World"); // создаем текстовый
узел
    const header.appendChild(headerText); // добавляем в элемент h1 текстовый узел
    document.body.appendChild(const header); // добавляем элемент h1 на страницу в
элемент body
  </script>
</body>
</html>

```

Сначала создаем обычный элемент заголовка h2 и текстовый узел. Затем текстовый узел добавляем в элемент заголовка. Затем заголовок добавляем в элемент **body**:

Стоит отметить, что нам необязательно для определения текста внутри элемента создавать дополнительный текстовый узел, так как мы можем воспользоваться свойством **textContent** и напрямую ему присвоить текст:

```

const header = document.createElement("h1"); // создаем заголовок <h1>
header.textContent = "Hello World"; // определяем текст элемента

```

В этом случае текстовый узел будет создан неявно при установке текста.

insertBefore

Метод `appendChild()` добавляет элемент в конец контейнера. Чтобы более конкретизировать место для добавления, можно использовать другой метод - **insertBefore()**, который добавляет один элемент перед другим элементом. Например, у нас есть следующая страница:

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Исходный</title>
</head>
<body>
  <p>Text 1</p>
  <p>Text 2</p>
</body>
</html>

```

Допустим, нам надо добавить в элемент `body` перед первым параграфом заголовок. Можно сделать это так:

```

<!DOCTYPE html>

```

```

<html>
<head>
  <meta charset="utf-8" />
  <title>Добавление элемента (insertBefore)</title>
</head>
<body>
  <p>Text 1</p>
  <p>Text 2</p>
  <script>
    const header = document.createElement("h1");           // создаем заголовок <h1>
    header.textContent = "Page Header"; // определяем текст элемента
    // получаем первый параграф
    const firstP = document.body.firstElementChild;
    // добавляем элемент h1 перед параграфом firstP
    document.body.insertBefore(header, firstP);
  </script>
</body>
</html>

```

Если нам надо вставить новый узел на второе, третье или любое другое место, то нам надо найти узел, перед которым надо вставлять, с помощью комбинаций свойств `firstElementChild/lastElementChild` и `nextSibling/previousSibling`.

Копирование элемента

Иногда элементы бывают довольно сложными по составу, и гораздо проще их скопировать, чем с помощью отдельных вызовов создавать из содержимое. Для копирования уже имеющихся узлов у объекта `Node` можно использовать метод **`cloneNode()`**:

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Копирование элемента</title>
</head>
<body>
  <div id="article">
    <h1>Home Page</h1>
    <p>Text 1</p>
    <p>Text 2</p>
  </div>
  <script>
    const article = document.getElementById("article");
    // получаем последний параграф
    const lastP = article.lastElementChild;
    // клонируем элемент lastP
    const newLastP = lastP.cloneNode(true);
    // изменяем текст
    newLastP.textContent = "Publication Date: 21/12/2025";
    // добавляем в конец элемента article
    article.appendChild(newLastP);
  </script>
</body>
</html>

```

В метод `cloneNode()` в качестве параметра передается логическое значение: если передается `true`, то элемент будет копироваться со всеми дочерними узлами; если передается `false` - то копируется без дочерних узлов. То есть в данном случае мы копируем узел со всем его содержимым и потом добавляем в конец элемента с `id="article"`.

Замена элемента

Для замены элемента применяется метод **`replaceChild(newNode, oldNode)`** объекта `Node`. Этот метод в качестве первого параметра принимает новый элемент, который заменяет старый элемент `oldNode`, передаваемый в качестве второго параметра.

В данном случае заменяем первый элемент - первый параграф заголовком `h2`:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Замена элемента</title>
</head>
<body>
  <div id="article">
    <p>Home Page</p>
    <p>Text 1</p>
    <p>Text 2</p>
  </div>
  <script>
    const article = document.getElementById("article");
    // находим узел, который будем заменять
    // пусть это будет первый элемент
    const oldNode = article.firstChild;
    // создаем новый элемент
    const newNode = document.createElement("h2");
    // определяем для него текст
    newNode.textContent = "Hello World";
    // заменяем старый узел новым
    article.replaceChild(newNode, oldNode);
  </script>
</body>
</html>
```

Удаление элемента

Для удаления элемента вызывается метод **`removeChild()`** объекта `Node`. Этот метод удаляет один из дочерних узлов:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Замена элемента</title>
</head>
<body>
  <div id="article">
    <p>Home Page</p>
    <p>Text 1</p>
```

```

        <p>Text 2</p>
    </div>
    <script>
    const article = document.getElementById("article");
    // находим узел, который будем заменять
    // пусть это будет первый элемент
    const oldNode = article.firstChild;
    // создаем новый элемент
    const newNode = document.createElement("h2");
    // определяем для него текст
    newNode.textContent = "Hello World";
    // заменяем старый узел новым
    article.replaceChild(newNode, oldNode);
    </script>
</body>
</html>

```

В данном случае удаляется первый параграф из блока div.

Удаление всех элементов

Иногда возникает необходимость удалить все элементы. Для этого перебираем все элементы контейнера и удаляем их:

```

<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Замена элемента</title>
</head>
<body>
    <div id="article">
        <h1>Home Page</h1>
        <p>Text 1</p>
        <p>Text 2</p>
    </div>
    <script>
    const article = document.getElementById("article");
    while(article.firstChild){
        article.removeChild(article.firstChild);
    }
    </script>
</body>
</html>

```

Самостоятельная работа

Создайте на web-странице, **пятнадцать HTML-элементов** произвольного содержания. Удалите элемент, соответствующий номеру в подгруппе. С помощью комбинаций свойств `firstElementChild/lastElementChild` и `nextSibling/previousSibling` вставьте вместо удаленного элемент `<p>Фамилия Имя</p>`, замените его на элемент `<h3>Имя Фамилия</h3>` и скопируйте в начало.